

《计算机系统基础》

第7章 链接

授课老师:

何璐璐

luluhe@whu.edu.cn

通知：第3章随堂测试

■ 时间

- 周一班：5月6日
- 周四班：5月9日

■ 范围：

- 第三章内容

■ 题型：

- 单选，多选，填空

是否遇到过如下的错误信息？

```
输出
显示输出来源(S): 生成
1>----- 已启动生成: 项目: Test, 配置: Release Win32 -----
1> TestDlg.cpp
1>TestDlg.obj : error LNK2001: 无法解析的外部符号 _matDLLInitialize
1>E:\US2015_Work\Test\Release\Test.exe : fatal error LNK1120: 1 个无法解析的外部命令
===== 生成: 成功 0 个, 失败 1 个, 最新 0 个, 跳过 0 个 =====
```

```
输出
显示输出来源(S): 生成
1>----- 已启动生成: 项目: ConsoleApplication10, 配置: Debug x64 -----
1>l1.c
1>LINK : fatal error LNK1104: 无法打开文件 "kernel32.lib"
1>已完成生成项目 "ConsoleApplication10.vcxproj" 的操作 - 失败。
===== 生成: 成功 0 个, 失败 1 个, 最新 0 个, 跳过 0 个 =====
```



主要内容

- 链接
- 案例研究：库打桩/插桩 (Library interpositioning)

主要学习目标

- 理解链接器将帮助你构造大型程序
- 理解链接器将帮助你避免一些危险的编程错误
- 理解链接将帮助你理解语言的作用域规则是如何实现的
- 理解链接将帮助你理解其他重要的系统概念
- 理解链接将使你能夠利用共享库

回顾：一个典型程序的转换处理过程

经典的 “hello.c” C-源程序

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
}
```

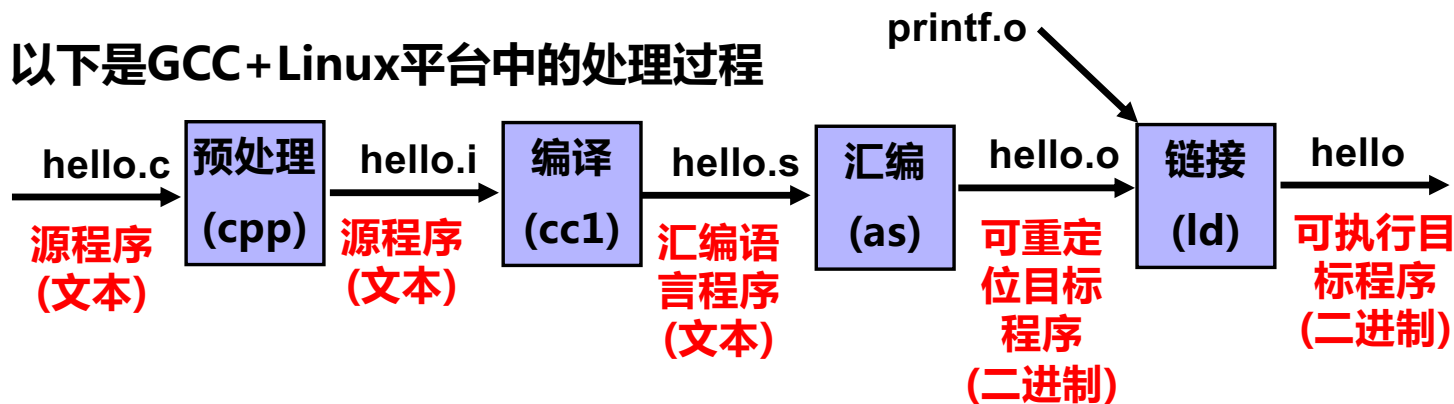
hello.c的ASCII文本表示

```
# i n c l u d e < s p > < s t d i o .
35 105 110 99 108 117 100 101 32 60 115 116 100 105 111 46
h > \n \n i n t < s p > m a i n ( ) \n {
104 62 10 10 105 110 116 32 109 97 105 110 40 41 10 123
\n < s p > < s p > < s p > < s p > p r i n t f ( " h e l
10 32 32 32 32 112 114 105 110 116 102 40 34 104 101 108
l o , < s p > w o r l d \n " ) ; \n }
108 111 44 32 119 111 114 108 100 92 110 34 41 59 10 125
```

功能：输出 “hello,world”

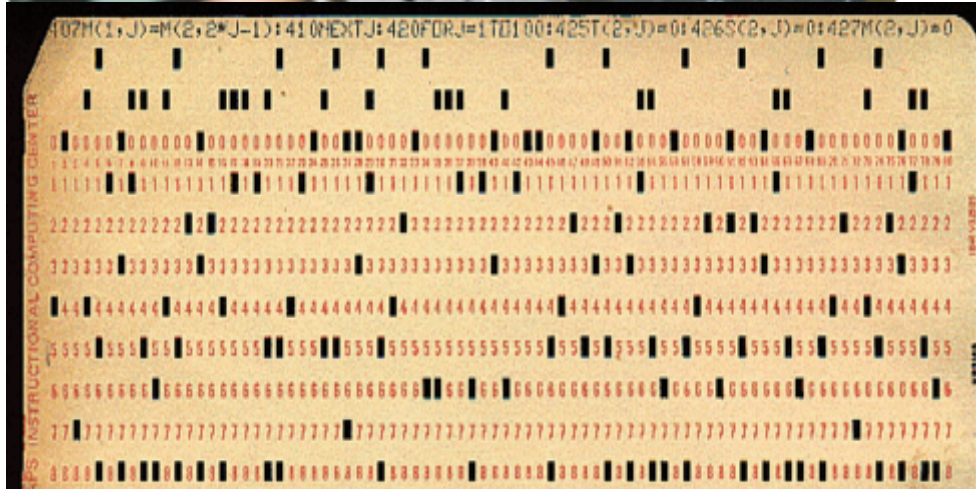
计算机不能直接执行hello.c！

以下是GCC+Linux平台中的处理过程



链接器的由来

■ 用机器语言编写程序，并记录在纸带或卡片上



输入：按钮、开关；所有信息都是0/1序列！
输出：指示灯等

假设：0010-jxx 转移指令

0 : 0101 0110

1 : 0010 0100

2 :

3 :

4 : 0110 0111

5 :

6 :

若在第5条指令前加入指令，则程序员需重新计算jmp指令的目标地址（重定位），然后重新打孔。

怎么办？

“Any problem in computer science can be solved with another level of indirection”.

by David Wheeler (剑桥大学计算机科学家)

用符号表示而不用0/1表示！

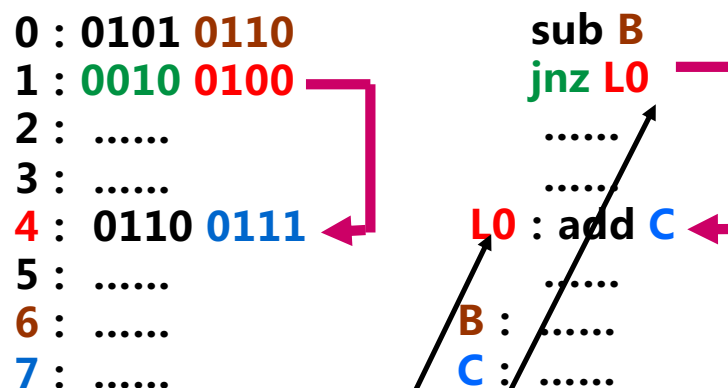
链接器的由来

■ 若用**符号**表示跳转位置和变量位置，是否简化了问题？

■ 于是，汇编语言出现

- 用助记符表示操作码
- 用**符号**表示位置
- 用助记符表示寄存器
- ...

0 : 0101 0110	sub B
1 : 0010 0100	jnz L0
2 :	
3 :	
4 : 0110 0111	L0 : add C
5 :	
6 :	B :
7 :	C :



更高级编程语言出现

程序越来越复杂，需多人开发不同的程序模块

子程序（函数）起始地址和变量起始地址是**符号定义**（definition）

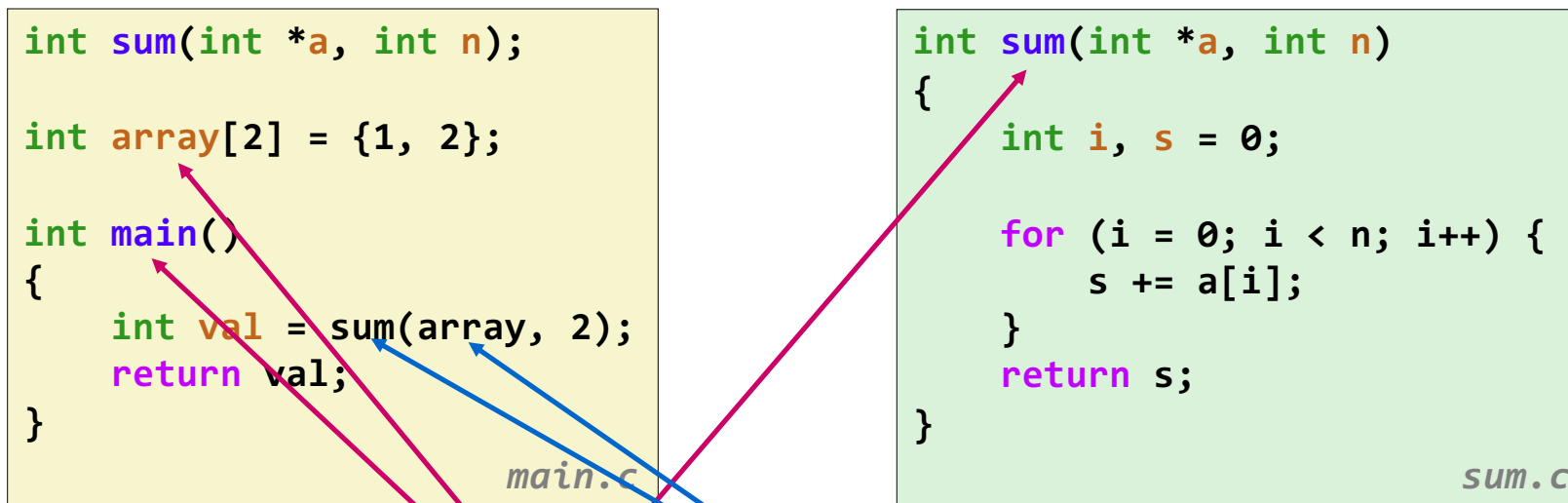
调用子程序（函数或过程）和使用变量即是**符号的引用**（reference）

一个模块定义的符号可以被另一个模块引用

最终须链接（即合并），合并时须在符号引用处填入定义处的地址

如上例，先确定L0的地址，再在jmp指令中填入L0的地址

C程序示例



你能说出哪些是符号定义？哪些是符号的引用？

局部变量分配在栈中，不会在过程外被引用，因此不是符号定义

静态链接

■ 用编译器翻译和链接程序:

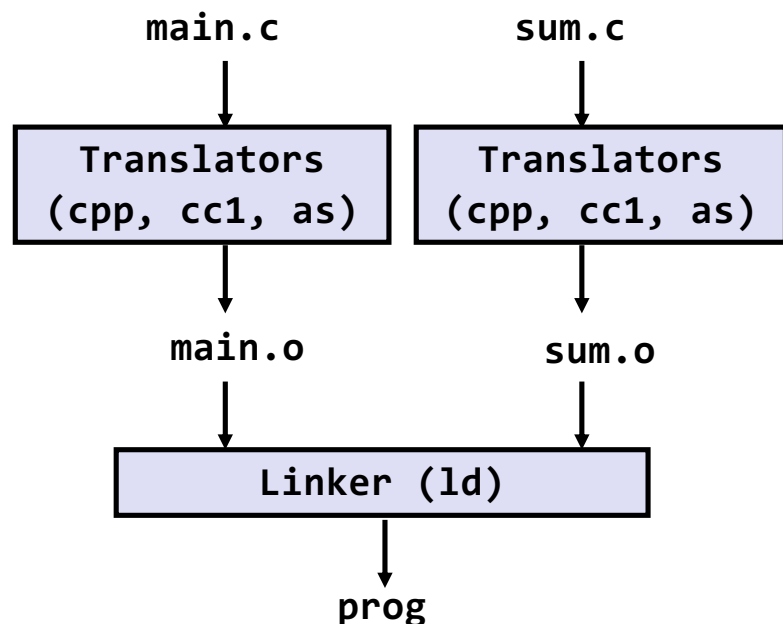
```
linux> gcc -Og -o prog main.c sum.c
linux> ./prog
```

```
cpp [other arguments] main.c /tmp/main.i
```

```
cc1 /tmp/main.i -Og [other arguments] -o /tmp/main.s
```

```
as [other arguments] -o /tmp/main.o /tmp/main.s
```

```
ld -o prog [system object file and args] /tmp/main.o /tmp/sum.o
```



源文件

预处理 **pre-processing (.i)**

编译 **compiling (.s)**

汇编 **assembling (.o)**

分别编译得到的

可重定位目标文件 (**relocatable object files**)

完全链接的

可执行目标文件 (**executable object file**)

(包含**main.c**和**sum.c**中定义的所有函数的代码和数据)

为什么要用链接器？

■ 理由1：模块化

- 可以用一系列小的源文件来完成程序的功能，而不是一个超级大的文件
- 可以将公共的函数做成库（后面会谈到）
 - 例如，数学库，标准C库

为什么要用链接器？（续）

■ 理由2：效率

- 时间：可以分开编译
 - 修改一个源文件，编译，再重链接
 - 不需要重编译其他源文件
- 空间：无需包含共享库的所有代码
 - 公共的函数可以打包进一个文件里
 - 而可执行文件和运行时内存镜像中只包含它们实际使用的函数的代码

链接器做什么？

■ 第一步：符号解析 **symbol resolution**

- 程序定义和引用符号 *symbols* （全局变量和函数）
 - `void swap() {...}` `/* define symbol swap */`
 - `swap();` `/* reference symbol swap */`
 - `int *xp = &x;` `/* define symbol xp, reference x */`
- 符号定义（由汇编器）存储在目标文件的**符号表** (*symbol table*) 中
 - 符号表是一个 `structs` 的数组
 - 每个条目包括符号的名字、大小和位置
- 符号解析步骤中，链接器将每个**符号引用**与一个确定的**符号定义**关联起来

链接器做什么？（续）

■ 第二步：重定位 **relocation**

- 将分开的代码段和数据段合并成统一的段
- 将 .o 文件中符号的**相对位置**重定位到它们在可执行文件中最后的**绝对内存位置**
- 更新这些符号的引用，使它们反映对应的新位置

让我们再仔细看看这两个步骤

三类目标文件（模块）

■ 可重定位目标文件（.o 文件）

- 包含二进制代码和数据，其形式使得它可以与其他可重定位目标文件合并起来，形成一个可执行目标文件
 - 每个 .o 文件由一个源文件（.c）产生得到
 - 每个.o文件代码和数据地址都从0开始

■ 可执行目标文件（a.out 文件）

- 包含二进制代码和数据，其形式可使得它被直接复制到内存并执行
- 代码和数据地址为虚拟地址空间中的地址

■ 共享目标文件（.so 文件）

- 一种特殊类型的可重定位目标文件，可以在加载时或运行时被动态地加载进内存并链接
- 在Windows上叫作动态链接库（*Dynamic Link Libraries* (DLLs)）


```

/* main.c */
int add(int, int);
int main( )
{
    return add(20, 13);
}

```

```

/* test.c */
int add(int i, int j)
{
    int x = i + j;
    return x;
}

```

objdump -d test.o 可重定位目标文件

00000000	<add>:	
0:	55	push %ebp
1:	89 e5	mov %esp, %ebp
3:	83 ec 10	sub \$0x10, %esp
6:	8b 45 0c	mov 0xc(%ebp), %eax
9:	8b 55 08	mov 0x8(%ebp), %edx
c:	8d 04 02	lea (%edx,%eax,1), %eax
f:	89 45 fc	mov %eax, -0x4(%ebp)
12:	8b 45 fc	mov -0x4(%ebp), %eax
15:	c9	leave
16:	c3	ret

objdump -d test 可执行目标文件

080483d4	<add>:	
80483d4:	55	push %ebp
80483d5:	89 e5	mov %esp, %ebp
80483d7:	83 ec 10	sub \$0x10, %esp
80483da:	8b 45 0c	mov 0xc(%ebp), %eax
80483dd:	8b 55 08	mov 0x8(%ebp), %edx
80483e0:	8d 04 02	lea (%edx,%eax,1), %eax
80483e3:	89 45 fc	mov %eax, -0x4(%ebp)
80483e6:	8b 45 fc	mov -0x4(%ebp), %eax
80483e9:	c9	leave
80483ea:	c3	ret

ELF 格式 (Executable and Linkable Format)

- 目标文件的标准二进制格式
- 下面这些文件都采用统一的格式
 - 可重定位目标文件 (.o),
 - 可执行目标文件 (a.out)
 - 共享目标文件 (.so)
 - core dumps
- 通用名字: ELF 二进制文件

ELF 目标文件格式

魔数：文件开头几个字节通常用来确定文件的类型或格式

ELF格式的魔数：7F 45 4C 46 （16进制）

加载或读取文件时，可用魔数确认文件类型是否正确

■ ELF 头部

- 字长、字节序、文件类型（.o, exec, .so）、机器类型等

```
root@cg:~/Desktop/test# readelf -h test1
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                        0
  Type:                               DYN (Shared object file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:                0x540
  Start of program headers:           64 (bytes into file)
  Start of section headers:          6448 (bytes into file)
  Flags:                              0x0
  Size of this header:                 64 (bytes)
  Size of program headers:            56 (bytes)
  Number of program headers:          9
  Size of section headers:            64 (bytes)
  Number of section headers:          29
  Section header string table index: 28
```

ELF header
Program header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
Section header table

ELF 目标文件格式

■ ELF 头部

- 字长、字节序、文件类型 (.o, exec, .so)、机器类型等

■ 程序头部表 **program header table**

- 页大小、虚拟地址内存段 (节, sections)、段大小

■ .text 节

- 代码节, 可读/可执行

■ .rodata 节

- 只读数据: 跳转表

■ .data 节

- 初始化了的全局变量, 可读/可写

■ .bss 节

- 未初始化或初始化为0的全局和静态变量
- 可读/可写
- “Block Storage Start”
- “Better Save Space”
- 具有节头部, 但是实际不占用空间

ELF header
Program header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
Section header table

0

ELF 目标文件格式（续）

■ .symtab 节

- 符号表
- 过程和静态变量的名字
- 节名字和位置

■ .rel.text 节

- .text 节的重定位信息
- 可执行文件中待修改指令的地址
- 待修改的指令

■ .rel.data 节

- .data 节的重定位信息
- 合并的可执行文件中待修改的指针数据的地址

■ .debug 节

- 有关符号调试的信息 (gcc -g)

■ 节头部表 section header table

- 每个节的偏移量和大小

ELF header
Segment header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
Section header table

0

符号和符号解析

- 每个可重定位目标模块m都有一个符号表，它包含了在m中定义和引用的所有符号。有三种链接器符号：
 - 全局符号
 - 由模块 m 定义，能够被其他模块引用的符号
 - 例如：非静态函数和非静态全局变量
 - 外部符号
 - 模块 m 引用的由其他模块定义的全局符号
 - 局部符号
 - 由模块 m 独占地定义和引用的符号
 - 例如：以 `static` 属性定义的 C 函数和全局变量
 - 链接器局部符号不是程序的局部变量（后者分配在栈中）

利用static属性隐藏变量和函数名字

步骤1：符号解析

```
int sum(int *a, int n);  
int array[2] = {1, 2};  
  
int main()  
{  
    int val = sum(array, 2);  
    return val;  
}  
main.c
```

在此定义

引用一个全局符号

定义一个全局符号

引用一个全局符号

链接器完全不知道 **val** 的存在

```
int sum(int *a, int n)  
{  
    int i, s = 0;  
    for (i = 0; i < n; i++) {  
        s += a[i];  
    }  
    return s;  
}  
sum.c
```

链接器完全不知道 **i** 或者 **s** 的存在

在此定义

符号解析-练习

全局符号

main.c

```
int buf[2] = {1, 2};
void swap();

int main()
{
    swap();
    return 0;
}
```

外部符号

全局符号

外部符号

局部符号

swap.c

```
extern int buf[];

int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
    int temp;

    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```

全局符号

局部变量 (非符号)

目标文件中的符号表

- 符号表 (symtab) 中每个条目的结构如下 (64位系统)

```
typedef struct {  
    int    name;      /*指向符号对应字符串在strtab节中的偏移*  
    char   type: 4,    /*符号对应目标的类型：数据、函数、源文件、节*/  
                binding: 4; /*符号对应目标是全局符号还是局部符号*/  
    char   reserved;  
    short  section;    /*符号对应目标所在的section，或其他情况*/  
    int    value;       /*在对应section中的偏移量，可执行文件中是虚拟地址*/  
    int    size;        /*符号对应目标所占字节数*/  
} Elf64_Symbol;
```

readelf -s <目标文件名>

其他情况：ABS表示不该被重定位；UND表示未定义；COM表示未初始化数据（.bss），此时，value表示对齐要求，size给出最小长度

READELF用一个整数索引来标识每个节。
Ndx=1表示.text节，而Ndx=3表示.data节。

■ readelf -s main.o

链接器内部使用的局部变量

Symbol table '.symtab' contains 11 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	symbol.c
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	SECTION	LOCAL	DEFAULT	4	
5:	0000000000000000	0	SECTION	LOCAL	DEFAULT	6	
6:	0000000000000000	0	SECTION	LOCAL	DEFAULT	7	
7:	0000000000000000	0	SECTION	LOCAL	DEFAULT	5	
8:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	3	array
9:	0000000000000000	31	FUNC	GLOBAL	DEFAULT	1	main
10:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	sum

大小

对象

函数

不确定

.data段

未定义

.text段

练习题7.1

这个题目针对图 7.5 中的 `m.o` 和 `swap.o` 模块。对于每个在 `swap.o` 中定义或引用的符号，请指出它是否在模块 `swap.o` 中的 `.symtab` 节中有一个符号表条目。如果是，请指出定义该符号的模块（`swap.o` 或者 `m.o`）、符号类型（局部、全局或者外部）以及它在模块中被分配到的节（`.text`、`.data`、`.bss`或COMMON）。

(a) m.c

code/link/m.c

```
1 void swap();
2
3 int buf[2] = {1, 2};
4
5 int main()
6 {
7     swap();
8     return 0;
9 }
```

(b) swap.c

code/link/swap.c

```
1 extern int buf[];
2
3 int *bufp0 = &buf[0];
4 int *bufp1;
5
6 void swap()
7 {
8     int temp;
9
10    bufp1 = &buf[1];
11    temp = *bufp0;
12    *bufp0 = *bufp1;
13    *bufp1 = temp;
14 }
```

Figure 7.5 Example program for Practice Problem 7.1.

符号	.symtab条目?	符号类型	在哪个模块中定义	节
buf				
bufp0				
bufp1				
swap				
temp				

练习题7.1

swap.o 的符号表

Symbol table '.symtab' contains 12 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	swap.c
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	SECTION	LOCAL	DEFAULT	5	
5:	0000000000000000	0	SECTION	LOCAL	DEFAULT	7	
6:	0000000000000000	0	SECTION	LOCAL	DEFAULT	8	
7:	0000000000000000	0	SECTION	LOCAL	DEFAULT	6	
8:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	3	bufp0
9:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	buf
10:	0000000000000008	8	OBJECT	GLOBAL	DEFAULT	COM	bufp1
11:	0000000000000000	60	FUNC	GLOBAL	DEFAULT	1	swap

(a) m.c

```
1 void swap();
2
3 int buf[2] = {1, 2};
4
5 int main()
6 {
7     swap();
8     return 0;
9 }
```

code/link/m.c

(b) swap.c

```
1 extern int buf[];
2
3 int *bufp0 = &buf[0];
4 int *bufp1;
5
6 void swap()
7 {
8     int temp;
9
10    bufp1 = &buf[1];
11    *bufp0;
12    = *bufp1;
13    = temp;
```

code/link/swap.c

gcc中：

COMMON: 未初始化的全局变量

.bss：未初始化的静态变量，以及初始化为0的全局或静态变量

code/link/swap.c

符号	.symtab条目?	符号类型	在哪个模块中定义	节
buf	是	GLOBAL	m.c	.data
bufp0	是	GLOBAL	swap.c	.data
bufp1	是	GLOBAL	swap.c	COMMON
swap	是	GLOBAL	swap.c	.text
temp	否	-	swap.c	-

符号解析

- 目的：将每个模块中引用的符号与某个目标模块中的定义符号建立关联。
- “符号的定义”其实质是什么？
- 是指符号被分配了虚拟地址空间。
 - 符号为函数名：指其代码所在区；符号为变量：指其占的静态数据区。
- 当每个定义符号在代码段或数据段中都被分配了存储空间后，将引用符号与对应定义符号建立关联，就可在重定位时将引用符号的地址重定位为相关联的定义符号的地址。
- 局部符号在本模块内定义并引用，因此，其解析较简单，只要与本模块内唯一的定义符号关联即可。
- 全局符号（外部定义的、内部定义的）的解析涉及多个模块，故较复杂

符合解析错误

```
void foo(void);  
  
int main(){  
    foo();  
    return 0;  
}
```

```
linux> gcc Wall -Og -o linkerror linkerror.c  
/tmp/ccSz5uti.o: In function 'main':  
/tmp/ccSz5uti.o(.text+0x7): undefined reference to 'foo'
```

局部符号

■ 局部非静态 C 变量 vs. 局部静态 C 变量

- 局部非静态 C 变量：存储在栈上
- 局部静态 C 变量：存储在 `.bss` 或 `.data` 中

```
int f()
{
    static int x = 0;
    return x;
}

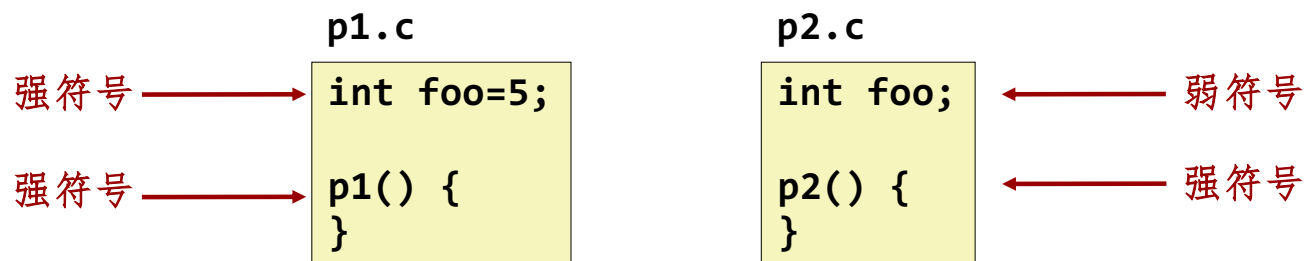
int g()
{
    static int x = 1;
    return x;
}
```

编译器为 `x` 的每个定义都在 `.data` 中分配空间

在符号表中为每个局部符号创建不同的、唯一的名字，例如 `x.1` 和 `x.2`

链接器如何解析多个同名符号的定义

- 程序符号分为强（**strong**）和弱（**weak**）两种
 - 强符号：过程和初始化了的全局符号，视为定义
 - 弱符号：未初始化的全局符号，可视为声明



变量的声明与定义

变量声明 (Variable Declaration):

变量声明是告诉编译器有一个变量存在，但并没有为其分配内存空间。它通常包括变量的类型和名称。

示例: `extern int x;` 表示存在一个整数类型的变量 `x`，但并没有分配内存空间。`extern` 关键字表示该变量在其他地方定义。

变量定义 (Variable Definition):

变量定义是在声明的基础上为变量分配内存空间，同时可以给变量赋初值。

示例: `int x = 10;` 表示定义了一个整数类型的变量 `x`，并分配了内存空间，并将其初始化为10。

```
// 变量声明  
extern int x;
```

```
// 变量定义  
int y = 20;
```

```
int main() {  
    // 在这里可以使用 x 和 y  
    x = 5;  
    return 0;  
}
```

在实际编程中，通常将变量的声明放在头文件中，而将变量的定义放在源文件中。这有助于在多个文件中共享变量，同时避免重复定义。

链接器的符号解析规则

- 规则1：不允许有多个同名的强符号
 - 每个强符号只能被定义一次
 - 否则：报链接错误
- 规则2：如果有一个强符号和多个弱符号，选择强符号
 - 对弱符号的引用会被解析成强符号
- 规则3：如果有多个弱符号，任意选择一个
 - 可以用 `gcc -fno-common` 来覆盖该规则（现在是默认选项了）

为什么要有 **Common** 和 **.bss** 两种类型？

gcc中：

- **COMMON**：未初始化的全局变量
- **.bss**：未初始化的静态变量，以及初始化为 **0** 的全局或静态变量
- 链接器允许多个模块定义同名的全局符号，编译器翻译时遇到一个弱全局符号（放到 **COMMON** 中），不能决定使用哪个定义，把决定权留给链接器

为什么未初始化的静态变量放到 **.bss** 中而不是 **COMMON** 中？

链接器谜题

```
int x;  
p1() {}
```

```
p1() {}
```

链接时错误：两个强符号 (**p1**)。

```
int x;  
p1() {}
```

```
int x;  
p2() {}
```

对 **x** 的引用会被指向同一个未被初始化的 **int** ，
这真的是你期望的吗？

```
int x;  
int y;  
p1() {}
```

```
double x;  
p2() {}
```

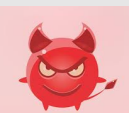
在 **p2** 中对 **x** 的写可能会覆盖 **y** !



```
int x=7;  
int y=5;  
p1() {}
```

```
double x;  
p2() {}
```

在 **p2** 中对 **x** 的写可能会覆盖 **y** !



```
int x=7;  
p1() {}
```

```
int x;  
p2() {}
```

对 **x** 的引用会指向同一个初始化了的变量。

最可怕的场景：两个相同的弱符号结构体定义，被两个具有不同对齐规则的编译器编译.....

多重定义符号的解析示例1

以下程序会发生链接出错吗？

```
int x=10;
int p1(void);
int main()
{
    x=p1();
    return x;
}
```

main.c

```
int x=20;
int p1()
{
    return x;
}
```

p1.c

多重定义符号的解析示例2

以下程序会发生链接出错吗？

```
# include <stdio.h>
int  y=100;
int  z;
void p1(void);
int main()
{
    z=1000;
    p1( );
    printf("y=%d, z=%d\n", y, z);
    return 0;
}
```

main.c

```
int  y;
int  z;
void p1( )
{
    y=200;
    z=2000;
}
```

p1.c

问题：打印结果是什么？

多重定义符号的解析举例3

以下程序会发生链接出错吗? `main.c`

```
1  #include <stdio.h>
2  int d=100;
3  int x=200;
4  void p1(void);
5  int main()
6  {
7      p1();
8      printf("d=%d,x=%d\n",d,x);
9      return 0;
10 }
```

`p1.c`

```
1  double d;
2
3  void p1()
4  {
5      d=1.0;
6  }
```

p1执行后d和x处内容是什么?

问题: 打印结果是什么?

全局变量

- 如果可以，请尽量避免
- 否则
 - 如果可以，请使用 **static**
 - 如果你定义了一个全局变量，请初始化
 - 如果你引用了外部的全局变量，请使用 **extern**

练习

假设一个C 程序有两个源文件：main.c 和test. c ，内容如下：

```
1  /* main.c */
2  int sum();
3
4  int a[4]={1, 2, 3, 4};
5  extern int val;
6  int main( )
7  {
8      val=sum();
9      return val;
10 }
```

```
1  /* test.c */
2  extern int a[];
3  int val=0;
4  int sum()
5  {
6      int i;
7      for (i=0; i<4; i++)
8          val += a[i];
9      return val;
10 }
```

对于编译生成的可重定位目标文件test.o ，填写下表中各符号的情况。

符号	是否在test.o的符号表中	符号类型（全局、局部、外部）	符号定义所在的模块及节
a			
val			
sum			
i			